

INF-2201

06 – Processes, Threads, Scheduling, Preemption

John Markus Bjørndalen
2026

Topic: Processes and multitasking

- Work in progress
 - Reorganizing lectures to match projects (moving target)

Problem: computer can only run one program simultaneously

- Computers used to be very expensive
- First thought: Sometimes the CPU is just waiting for I/O (user input, disks, paper tapes, somebody to load the tapes in the reader, ...)
 - What if you could have some other program running simultaneously that could do things in the background?
- Second thought: if you can run multiple programs at the same time, then multiple users can share the same computer

Idea: multitasking

- Multitasking: let a computer run multiple tasks concurrently
 - If only one CPU:
 - switch between tasks often enough to give the illusion of running them at the same time
 - Switch to a different task when
 - one is blocked / waiting for something
 - when the current task has run for some time and we need to let other tasks make some progress

Observation

What does the program use and keep track of for a running program?

Basic information (for the current simple computer):

- Instruction pointer (IP/EIP)
- Stack pointer
- Registers

How they are used

- Variables are typically stored on heap (global variables or allocated variables) or stack/registers (local variables). NB: assuming a C-like language.
- Progress of the program is tracked using the IP
- Function calls:
 - Store state you need to keep (ex: registers) on the stack
 - Store parameters to function on stack or in registers (here: register A)
 - Store return pointer on the stack (the next instruction / address after the "call")
 - Jump/Call to the function
 - Function looks at parameters and does its things
 - Return value from function may be stored in a register (can also use stack, but slightly more tricky)
 - When returning back to caller: pop the stored instruction pointer into current IP (ret, iret, ...)
- Control returns to the calling function

; Super simple program in z80-ish asm

...

0d00 push b ; store registers

0d01 push c ; store registers

0d02 mov a, 42 ; set parameter

0d04 push 0d0a ; return addr/ip

0d07 call 0f00 ; call foo

0d0a pop c ; return here

0d0b pop b

.....

; "function foo"

0f00 inc a ; do something

... ; do something

0f40 mov a, 90 ; set return value

0f42 ret ; return / pop ip

Tracing the instructions
executed draws a
"thread" of execution
through the program

Observation

Basic information (for the current simple computer):

- Instruction pointer (IP/EIP)
- Stack pointer
- Registers

How they are used

- Sufficient to keep track of the current state of a running program in the current system we have!
- Store much of them when calling a function
- What if we can store everything we need before calling a "super function" that switches to a different program?
 - All registers
 - SP
 - IP

; Super simple program in z80-ish asm

```
...  
0d00 push b           ; store registers  
0d01 push c           ; store registers  
0d02 mov a, 42        ; set parameter  
0d04 push 0d0a         ; return addr/ip  
0d07 call 0f00         ; call foo  
0d0a pop c             ; return here  
0d0b pop b  
.....
```

; "function foo"

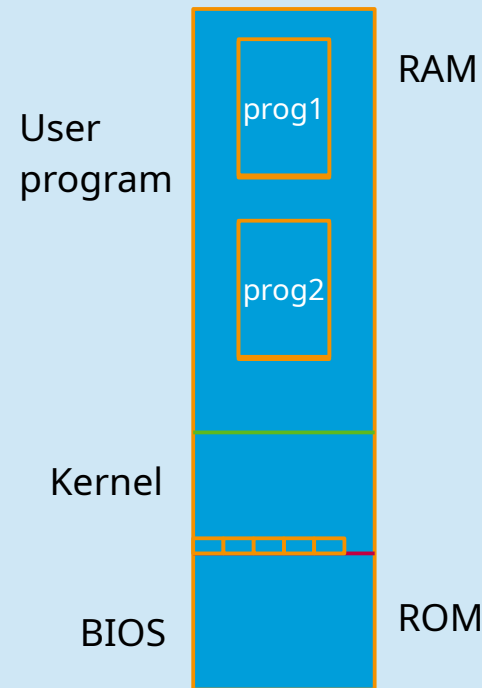
```
0f00 inc a             ; do something  
...                   ; do something  
0f40 mov a, 90         ; set return value  
0f42 ret               ; return / pop ip
```



Tracing the instructions
executed draws a
"thread" of execution
through the program

Running multiple programs

- Switch to another program (**yield**):
 - store/snapshot current state (all registers, including IP to "return here" and SP)
 - call function that selects which program gets to run next (let's call that the scheduler)
 - Restore registers (start with stack pointer)
 - "Return" to selected/scheduled program using "ret"
- NB: order of store and restore is important. One method:
 - Store IP of where to return to
 - Store registers
 - Store SP "somewhere"
- When restoring state
 - Restore SP (to pick the right stack)
 - Restore registers
 - "ret" to return to the indicated return position (instruction after the call to yield)



Processes

We currently have

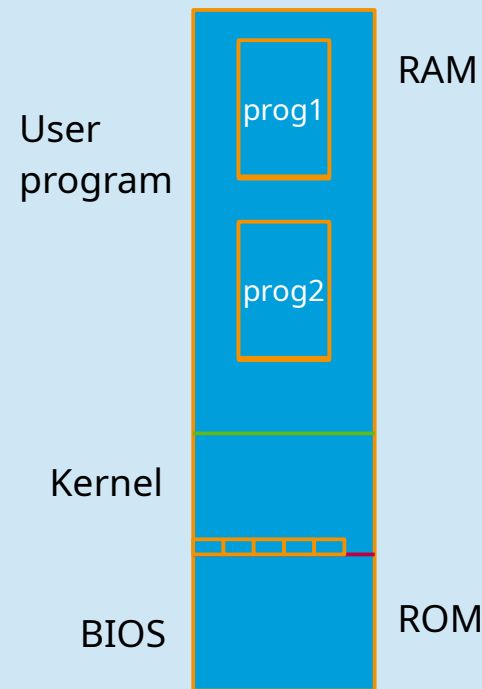
- Method for yielding execution to let somebody else take over: yield
- Method for selecting the next to run: scheduler

This is the first step to what we call **cooperative multitasking**

- Multiple programs that run concurrently, yielding execution time to each other

But how do we pick the next to run (**scheduler**), and where do we store the state of the programs?

- Introduce processes:
 - A process is a running program
 - The program binary
 - Data used by the program
 - Execution state (registers and stack)
 - Any other state that the operating system needs to keep track of the running process



Keeping track of processes

- Need information
 - Current execution state
 - When it is running: registers
 - When it is paused: kernels typically use a PCB (process control block) to save state necessary to restore the execution
 - Other state necessary for the kernel and processes
 - Identifier (ex: process id)
 - Permissions/right (we will look into this later)
- What is stored in a PCB? (Assume registers are stored on stack when yielding)
 - Process ID
 - Maybe memory location where the process is stored
 - Stack pointer (to let us restore/find stack and restore registers from there)
 - Some implementations might store a small kernel stack in the PCB to use for kernel functionality

```
struct process {
    struct file execfile;
    char    name[DEBUGSTR_MAX];

    pid_t    pid;
    uintptr_t start_addr;

    /* Stack addresses */
    uintptr_t ustack;
    uintptr_t kstack;

    /** Process's address space (page mapping hierarchy) */
    struct addrspc addrspc;

    /**
     * File descriptors
     *
     * - 0 = stdin
     * - 1 = stdout
     * - 2 = stderr
     */
    struct file *fds[FD_MAX];
};
```

Process struct from Mux P2 (2026)

Kernel needs to keep track of multiple processes

- If each process has a PCB that uniquely describes that process and state, use that PCB to also keep track of which processes that are currently waiting to run and which process is currently running
 - `current_process` → PCB of currently running process
 - `ready_queue` → queue of PCBs / processes waiting to run
- Process switching: when the **scheduler** is called:
 - Move the `current_process` PCB to the end of the ready queue
 - Move the first PCB from `ready_queue` to `current_process`
 - Restore process state of `current_process`
- This is called round robin scheduling
 - There are many options for more advanced scheduling algorithms, prioritizing things like interactivity, important tasks vs. less important tasks (priorities) etc.

How do we start a new process?

- We currently have mechanisms for yielding (storing state), selecting new state (schedule) and restore state.
- Instead of creating yet another mechanism, simply use the kernel's scheduler:
 - Create a restore state pointing to where you want the process to start
"fake" a process that was interrupted before running its first instruction
 - Fill in the PCB
 - Submit the process to the kernel scheduler
 - Let the scheduler run, pick and activate a process

Problem: starting and stopping new processes

How do we create new processes from user level (from a process)?

- "Parent" calls fork()
 - Makes a copy of the current process.
 - New process (child) gets a new PCB. Starts executing at the same instruction as the parent
 - Identifying which copy is the parent or child: Return value of fork() is
 - Parent: PID (Process ID) of child
 - Child: 0

This assumes Unix/POSIX API (other operating systems have similar mechanisms)

```
/* Source code from "main wait" on Ubuntu 22.04 */  
  
#include <sys/wait.h>  
#include <stdint.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <stdio.h>  
  
int main(int argc, char *argv[])  
{  
    pid_t cpid, w;  
    int wstatus;  
  
    cpid = fork();  
    if (cpid == -1) {  
        perror("fork");  
        exit(EXIT_FAILURE);  
    }  
  
    if (cpid == 0) {  
        /* Code executed by child */  
        printf("Child PID is %jd\n", (intmax_t) getpid());  
        if (argc == 1)  
            pause();  
        /* Wait for signals */  
        _exit(atoi(argv[1]));  
    } else {  
        /* Code executed by parent */  
        do {  
            w = waitpid(cpid, &wstatus, WUNTRACED | WCONTINUED);  
            if (w == -1) {  
                perror("waitpid");  
                exit(EXIT_FAILURE);  
            }  
  
            if (WIFEXITED(wstatus)) {  
                printf("exited, status=%d\n", WEXITSTATUS(wstatus));  
            } else if (WIFSIGNALED(wstatus)) {  
                printf("killed, status=%d\n", WTERMSIG(wstatus));  
            }  
        } while (w != 0);  
    }  
}
```

Problem: starting and stopping new processes

Waiting for a child process to terminate?

- Using one of the wait-calls (here: waitpid)

```
/* Source code from "main wait" on Ubuntu 22.04 */  
  
#include <sys/wait.h>  
#include <stdint.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <stdio.h>  
  
int main(int argc, char *argv[])  
{  
    pid_t cpid, w;  
    int wstatus;  
  
    cpid = fork();  
    if (cpid == -1) {  
        perror("fork");  
        exit(EXIT_FAILURE);  
    }  
  
    if (cpid == 0) {  
        /* Code executed by child */  
        printf("Child PID is %jd\n", (intmax_t) getpid());  
        if (argc == 1)  
            pause();  
        /* Wait for signals */  
        _exit(atoi(argv[1]));  
    } else {  
        /* Code executed by parent */  
        do {  
            w = waitpid(cpid, &wstatus, WUNTRACED | WCONTINUED);  
            if (w == -1) {  
                perror("waitpid");  
                exit(EXIT_FAILURE);  
            }  
  
            if (WIFEXITED(wstatus)) {  
                printf("exited, status=%d\n", WEXITSTATUS(wstatus));  
            } else if (WIFSIGNALED(wstatus)) {  
                printf("signaled, status=%d\n", WTERMSIG(wstatus));  
            }  
        } while (w != -1);  
    }  
}
```

Problem: child should run a different program

Sometimes need to run a different program in the child than in the parent

- Solution: call `exec()` in child after returning from `fork()`
- Replaces the program code (and data) in the current program with new code and data
 - Typically starts running from `main()`
 - Can be called at any point in time (not just after `fork()`)

Python `exec` example: replace Python process with `/bin/l`s

```
>>> import os
>>> os.execv("/bin/l", ["/bin/l", "/usr/local"])
bin etc games include lib man sbin share src
```



NOP

NOP

Problem: multiple activities in a single program

- Some system calls are blocking
 - "wait for user input"
 - "wait for file to be updated by another program"
- Each of these block would block the entire process
 - Ex: if program is waiting for file updates, it will not make any progress on user input
- Could support multiple activities by forking / using multiple processes
 - One process to handle each activity
- Several challenges:
 - Memory overhead (complete copy of program)
 - Communication overhead (processes need to coordinate, keep copies of shared state consistent, ...)
 - Context switch overhead (switch between processes)
 - Fork/join overhead

Solution: Threads

- Revisiting: thread of execution
- Like leaving a string to navigate a labyrinth (think Ariadne)
- What if we could have more than one (explorer)?



Image: Microsoft Copilot

```
; Super simple program in z80-ish asm
...
0d00  push b           ; store registers
0d01  push c           ; store registers
0d02  mov a, 42        ; set parameter
0d04  push 0d0a        ; return addr/ip
0d07  call 0f00        ; call foo
0d0a  pop c            ; return here
0d0b  pop b
.....
```

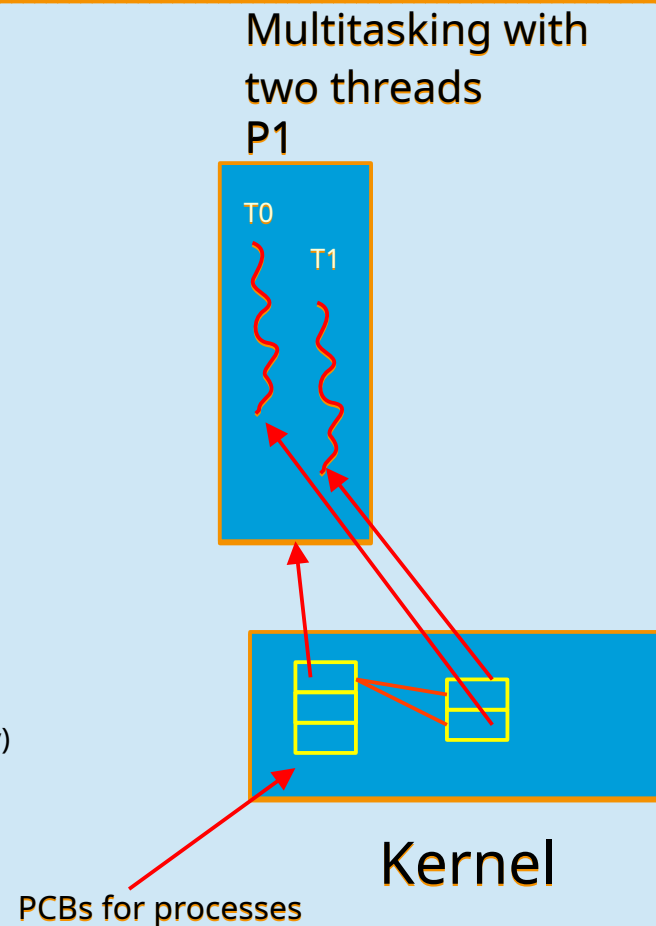
```
; "function foo"
0f00  inc a            ; do something
...
0f40  mov a, 90        ; set return value
0f42  ret              ; return / pop ip
```

Tracing the instructions
executed draws a
"thread" of execution
through the program

Solution: threads

Like processes, threads are used to keep track of execution. They differ in some ways:

- Parent and child thread share memory, open files etc.
- Use a different API (typical on Unix: pthreads), but similar concepts to process creation
- A thread is typically viewed as executing "inside" a process
 - One view: a process is a container that always has at least one default thread to track the execution. Can then add more threads.
 - Threads have individual Thread Control Blocks (TCB), but may share Process Control Blocks (PCB).
 - Less overhead for context switching
 - Some of it related to topics we haven't covered yet
 - Potentially less overhead for keeping shared state consistent (it's shared memory)
 - Some issues introduced though...



How do we run threads

We need

- some way of keeping track of each threads execution |
- Some mechanism for switching between threads
 - When do we switch?
- Mechanisms for starting, stopping and merging threads
 - One thread should be able to create more threads
 - The creator thread (parent) should be able to wait for the threads it created (child threads)
 - A thread should be able to finish / exit without killing other threads
 - What do we do when the last thread in a process exits?

Threads

Example: pthreads API

- Create a thread: `pthread_create`
 - Specify function to call and parameters to function
 - Where do you want the thread to start executing and what data should it get
 - Returns thread ID
- Wait for a thread: `pthread_join`
 - Specify thread ID
- What will this program print?

```
#include <pthread.h>
#include <stdio.h>

const int N = 10;

typedef struct {
    int th_no;
} thread_parm;

void* thread_proc(void *vargs)
{
    thread_parm *parm = (thread_parm*) vargs;
    printf("This is thread with param %d\n", parm->th_no);
    return NULL;
}

int main(int argc, char* argv[])
{
    pthread_t threads[N];
    thread_parm parms[N];

    // Create N threads.
    for (int i = 0; i < N; i++) {
        parms[i].th_no = i;
        pthread_create(&threads[i], NULL, thread_proc, &parms[i]);
    }
    printf("All spawned, now waiting for them to complete\n");
    for (int i = 0; i < N; i++) {
        pthread_join(threads[i], NULL);
    }

    return 0;
}
```

Threads

Example: pthreads API

```
>cc -Wall thread1.c
```

```
>./a.out
```

This is thread with param 0

This is thread with param 3

This is thread with param 4

This is thread with param 1

This is thread with param 2

This is thread with param 5

This is thread with param 6

This is thread with param 7

This is thread with param 8

All spawned, now waiting for them to complete

This is thread with param 9

The system is not required to run the threads in the order you created them!

```
#include <pthread.h>
#include <stdio.h>

const int N = 10;

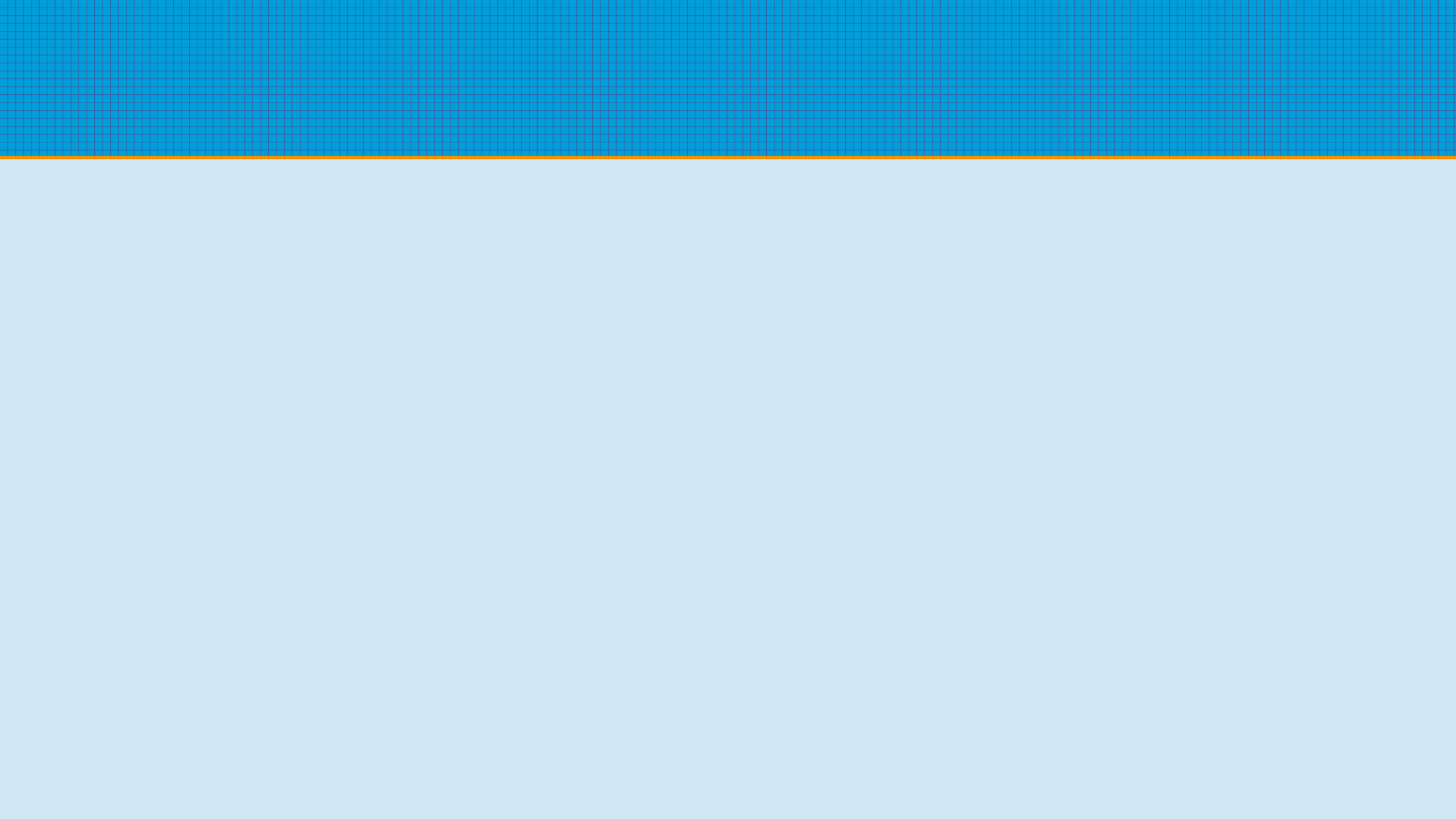
typedef struct {
    int th_no;
} thread_parm;

void* thread_proc(void *vargs)
{
    thread_parm *parm = (thread_parm*) vargs;
    printf("This is thread with param %d\n", parm->th_no);
    return NULL;
}

int main(int argc, char* argv[])
{
    pthread_t threads[N];
    thread_parm parms[N];

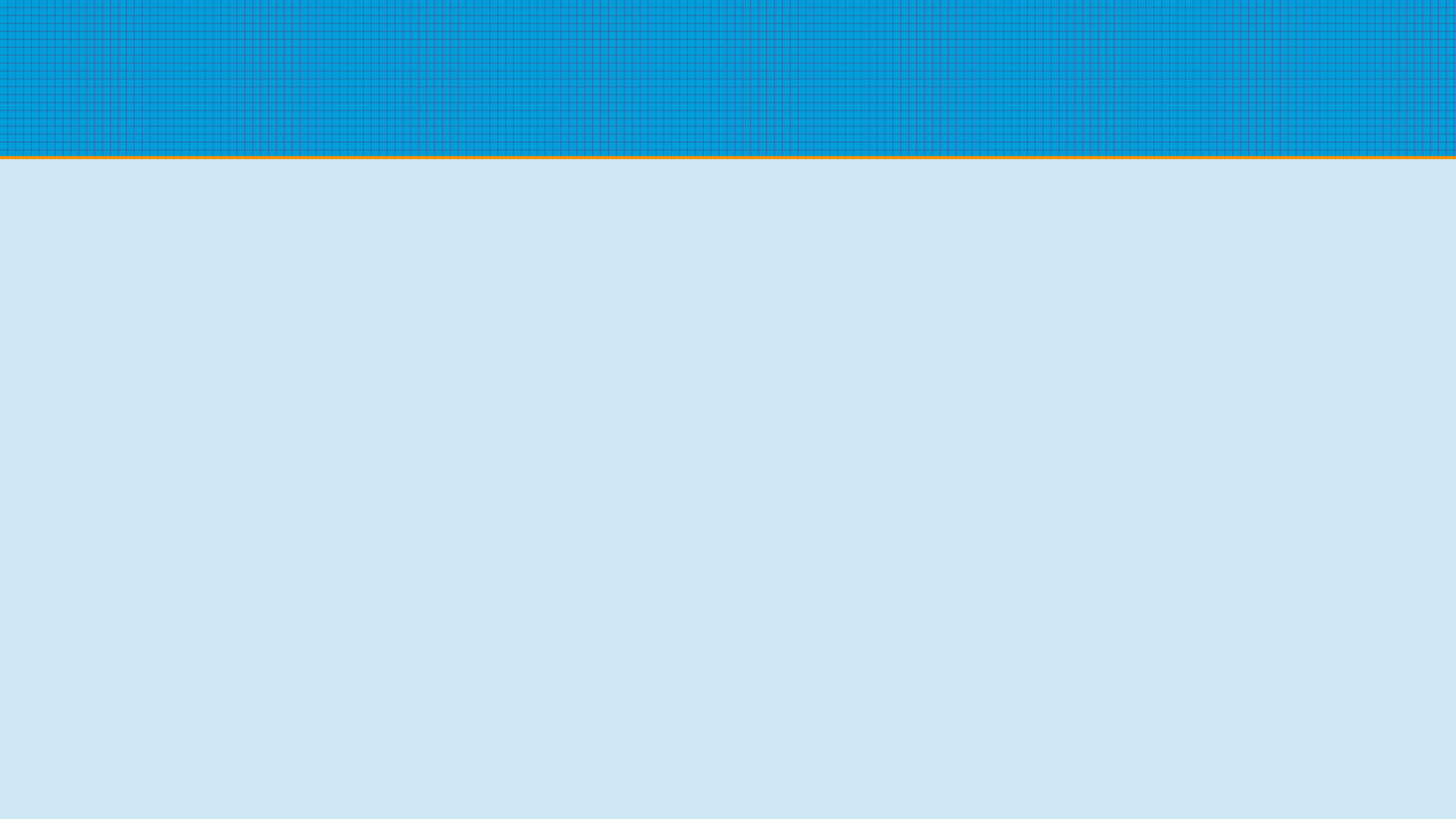
    // Create N threads.
    for (int i = 0; i < N; i++) {
        parms[i].th_no = i;
        pthread_create(&threads[i], NULL, thread_proc, &parms[i]);
    }
    printf("All spawned, now waiting for them to complete\n");
    for (int i = 0; i < N; i++) {
        pthread_join(threads[i], NULL);
    }

    return 0;
}
```



How to implement threads

- User level threads – example with cooperative multitasking
 - Thread switch with function call (yield)
 - Should we even bother with this? Time?
 - Create, keep track of, schedule..
 - Issue: blocking system calls.
 - See more: coroutines, user space threads / user level threads
- Kernel threads – in Unix we will use **preemptive multitasking**:
 - Managed by kernel, not execute in kernel (confusion from old pre-2026 code)
 - Switching threads: already have mechanisms we can re-use: system calls and interrupts require storing and restoring current user level execution.
 - Key idea: if a thread is paused due to interrupt, what if we restore another thread instead?
 - Which thread to restore: decision made by scheduler.
 - When do we switch to give illusion of concurrent execution? Clock interrupt.
 - Scheduler:
 - Selects the next process or thread to run
 - Policy can be simple (round robin / next in queue), or more advanced (priorities, fair share, ..., interactive vs. Batch, ...)
 - Preempt: key idea is that the process or thread can be interrupted at any point in the execution, and the scheduler can pick another thread to run
 - Timer interrupt



Side note

- Linux has more flexibility when creating threads or processes when using the low-level **clone** system call
 - <https://man7.org/linux/man-pages/man2/clone.2.html>
- Lets caller control which resources should be shared or not shared between the parent and child.

Whether it is a traditional process or thread depends on the set of flags specified when using clone.
-

Summary

- Multitasking
 - Cooperative vs. Preemptive multitasking
- Processes vs threads
- Scheduling
- Next week:
 - Concurrency, coordination, synchronization